

IS 2901 - Software Development Project
Valuation Document Management &
CRM System

CodeHub

Index No.	Name
235051C	G.C.G. Jagoda
235075D	S.W.P. Methmini
235104T	A.L.A.A. Shanali
235078M	H.M.R.R. Narampanawa
235030L	B.G.R. Dilsara

Supervised by:

Ms. Nipuni Premadasa

Ms. Shalini Upeksha

Client:

S. R. Galappaththy

ZAVOLT (PVT) LTD

Faculty of Information Technology

University of Moratuwa

2026

Declaration

We declare that this report is our own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published or unpublished work of others has been acknowledged in the text, and a list of references is given.

Names of Students	Signatures of Students
G.C.G. Jagoda	
S.W.P. Methmini	
A.L.A.A. Shanali	
H.M.R.R. Narampanawa	
B.G.R. Dilsara	

Supervised by:

Names of Supervisors	Signatures of Supervisors	Date
Ms. Nipuni Premadasa		
Ms. Shalini Upeksha		

Abstract

Valuation companies working with banks and financial institutions manage client information, property records, site inspection evidence, valuation reports, approvals, and payment-related communication. When these activities are handled through paper forms, spreadsheets, email, and telephone calls, information is repeatedly entered, work status is difficult to trace, and report preparation is delayed.

This project presents the **Valuation Document Management & CRM System**, a web-based platform that digitises the valuation lifecycle. The implemented system allows an applicant to submit a valuation request; a coordinator to register applicants and banks, create projects and valuations, and assign a technical officer; and a technical officer to record inspection data, upload photographs, select GPS coordinates, analyse nearby comparable land evidence, and prepare a draft report. The draft is reviewed sequentially by Manager Level 3, Manager Level 2, and Manager Level 1. After the final reviewer locks the report, the applicant completes payment or submits a payment slip for verification, and the requesting bank can access the locked and paid report.

The solution uses React 18, TypeScript, Vite, Tailwind CSS, React Router, and Leaflet for the frontend. NestJS and TypeScript provide the backend API, with PostgreSQL as the primary database. The application includes password hashing with bcrypt, email and in-system notifications, private object storage with a database fallback, OCR-assisted inspection entry, OpenStreetMap-based location support, and optional AI-assisted descriptive content. The project demonstrates how a modular role-based workflow can improve the visibility, consistency, and traceability of land-valuation administration while retaining human review for professional decisions.

Keywords: land valuation, document management, customer relationship management, workflow automation, report approval, OCR, GIS, React, NestJS, PostgreSQL.

Table of Contents

Chapter 1 - Introduction	1
1.1 Introduction	1
1.2 Problem in Brief	2
1.3 Aim and Objectives	3
1.4 Proposed Solution	4
1.5 Structure of the Report	5
Chapter 2 - Literature Review	6
Chapter 3 - Adopted Technologies and Approach	10
Chapter 4 - Analysis and Design	16
Chapter 5 - Implementation and Testing	25
Chapter 6 - Discussion and Conclusion	36
References	41
Appendix A - Individual Contribution	43
Appendix B - Evidence and Screenshot Checklist	45

Page numbers are indicative. Update the

table of contents in Microsoft Word after adding your final screenshots, diagrams, and signatures.

List of Figures and Tables

Figure 1 - End-to-end valuation workflow	17
Figure 2 - System architecture	19
Figure 3 - Report approval state model	22
Table 1 - User roles and responsibilities	12
Table 2 - Functional requirements	17
Table 3 - Technology stack	11
Table 4 - Test cases	33

Chapter 1 – Introduction

1.1 Introduction

Property valuation is a key activity in loan approval, property transactions, insurance, investment analysis, and asset management. A valuation firm must collect information about the client and property, arrange a site inspection, evaluate physical and legal evidence, prepare a valuation report, obtain managerial approval, and release the completed report to the correct party. These activities involve loan applicants, banks, coordinators, technical officers, managers, and administrators.

In the client context, much of this work is dependent on handwritten inspection records, separate documents, spreadsheets, calls, and emails. This makes it difficult to see a project's current stage, follow up missing work, preserve supporting evidence, and ensure that a report is released only after the correct approval and payment conditions have been met. A centralised application is therefore needed to coordinate the work while presenting each user with functions relevant to their responsibility.

1.2 Problem in Brief

Manual and fragmented valuation handling leads to duplicated data entry, inconsistent project information, limited visibility of assignments, delayed report preparation, and poor traceability of report revisions. Coordinators must repeatedly contact officers and clients to obtain status updates. Technical officers need to combine field notes, property details, photographs, maps, and market information while creating a professional report. Managers need a clear, controlled approval chain, while applicants and banks require timely but appropriately restricted status and report access.

1.3 Aim and Objectives

1.3.1 Aim

To design and develop a web-based Valuation Document Management & CRM System that centralises and streamlines the land valuation lifecycle through role-specific functions, structured valuation evidence, sequential report review, communication, payment handling, and controlled final-report access.

1.3.2 Objectives

1. Provide secure account login, password management, and role-focused navigation.
2. Enable public valuation requests and coordinator management of applicants, banks, projects, valuations, and technical-officer assignments.
3. Support capture of inspection data, supporting documents, photographs, GPS coordinates, locality descriptions, and comparable land evidence.
4. Prepare a structured report draft and implement L3, L2, and L1 approval and correction stages.
5. Support payment calculation, card demonstration payment, payment-slip verification, and post-payment bank report access.
6. Provide messages, notifications, email updates, and resilient file storage.

1.4 Proposed Solution

The proposed system is a two-tier web application. The React client provides public pages and internal portals for every role. A NestJS API implements the workflow and persists operational data in PostgreSQL. The technical-officer workflow combines inspection capture, OCR-assisted extraction from an uploaded inspection form, photo uploads, mapping, comparable evidence, value analysis, and a report editor. The report proceeds through a defined approval chain. At the final stage, payment state is checked before the requesting bank can view the report.

1.5 Structure of the Report

Chapter 2 reviews relevant concepts and existing approaches. Chapter 3 presents the adopted technologies and project approach. Chapter 4 describes requirements and the system design. Chapter 5 explains implemented modules and testing. Chapter 6 evaluates the solution, identifies limitations, and presents future improvements.

1.6 Summary

This chapter introduced the valuation-management problem, project goal, objectives, and proposed solution. The next chapter establishes the conceptual background for the system.

Chapter 2 – Literature Review

2.1 Introduction

This chapter reviews concepts relevant to the proposed solution: document management, customer relationship management, OCR and AI assistance, workflow automation, geographic information, and secure role-based systems. The review is used to position the project as an integrated operational system rather than a single-purpose report generator.

2.2 Document Management and CRM Systems

Document management systems centralise records, make documents easier to retrieve, and reduce dependence on personal folders and paper files. CRM systems manage customer details, communications, requests, and service history. In a valuation setting these functions overlap: the same project requires client identity, property documents, inspection evidence, bank details, communication, and an approved report. General document-management and CRM products provide useful principles, but normally require substantial customisation to reflect valuation-specific activities and approval rules.

2.3 OCR and AI-Assisted Data Entry

OCR converts text in scanned files into machine-readable output and can reduce repetitive data entry. However, handwritten forms, inconsistent formats, and low-quality scans require a human user to verify and correct extracted values. The system implements OCR as an assistance mechanism: an uploaded inspection form is sent to OCR.space and the returned text is parsed into an editable inspection draft. AI assistance is optional and is used for concise photo captions and evidence-grounded narrative text. The system provides template-based fallback text if AI services are not configured or fail.

2.4 Workflow and Approval Management

Workflow management creates explicit states, assigns responsibility at each step, and records the movement of work between users. This is important for professional reports because approval is not merely a notification; it is a controlled quality gate. The project uses a progression from draft to Manager L3, L2, and L1 review. A rejection returns the work to the appropriate earlier stage with a reason. A locked report becomes immutable in the normal workflow and enters the payment stage.

2.5 Mapping and Comparable Evidence

Land valuation depends strongly on location, accessibility, surrounding development, property characteristics, and available comparable evidence. Mapping tools help technical officers preserve the selected property position and prepare access/locality descriptions. The system uses Leaflet to present the map interface and can use OpenStreetMap Nominatim reverse geocoding as additional location context. Comparable evidence is stored with factors such as area, price per perch, source, evidence type, and distance, enabling a documented basis for market-value analysis.

2.6 Research Gap and Project Contribution

The reviewed areas show that each technology addresses part of the problem, but the operational need is an integrated workflow. This project combines customer/project handling, technical evidence capture, report preparation, a three-level approval chain, notifications, payment confirmation, and bank-facing report access in one domain-specific application. Professional judgement remains with the technical officer and managers; automation is designed to assist rather than replace that judgement.

2.7 Summary

The literature supports a system that combines structured data, controlled workflow, and human verification. The next chapter explains the technologies chosen to implement that system.

Chapter 3 – Adopted Technologies and Approach

3.1 Technologies Used

Technology	Use in the project	Justification
React 18, TypeScript and Vite	Single-page frontend and build tooling	Component reuse, type safety, fast development workflow
React Router and Tailwind CSS	Navigation and responsive interface styling	Role-specific pages and consistent reusable UI
Leaflet / react-leaflet	GPS map and location selection	Interactive open mapping in the technical-officer workflow
NestJS and TypeScript	REST API, business logic, feature modules	Modular server structure suitable for a multi-role system
PostgreSQL	Primary relational data store	Reliable transactions, structured data, and JSONB flexibility
Supabase Storage	Private object storage for files	Stores uploaded documents and photographs; application retains a fallback
bcryptjs and Nodemailer	Password hashing and email	Protects passwords and supports workflow communication
OCR.space, OpenStreetMap, optional Gemini API	OCR, map context, optional content assistance	Enhances data entry and location/narrative workflows while keeping user review

Table 1 - Technology stack and justification

3.2 User Roles, Inputs, Processes and Outputs

Role	Primary input/process	Output
Loan Applicant	Request, documents and payment/slip	Project status and payment confirmation
Coordinator	Applicant/bank registration, project/valuation creation, assignment	Managed projects, assignments and verified slips
Technical Officer	Inspection, photos, map, comparable evidence, draft	Structured valuation report draft
Managers L3/L2/L1	Review, correction request, approval/lock	Controlled report progression and final lock
Bank	Associated project lookup	Access to a locked, paid report
Administrator	Internal account management	Staff accounts and user information

Table 2 - User roles and responsibilities

3.3 Development Approach

The project was developed iteratively using an Agile approach. The team used a GitHub repository and project-board practices to divide work into features and track progress. The modular code structure allowed parallel work across coordinator, technical-officer, client/bank, manager, administration, and shared services. The final implementation was checked through builds and end-to-end workflow scenarios.

3.4 System Working Behaviour

The frontend operates as a React single-page application. It sends `/api` requests to the NestJS server, which applies feature-specific business logic and accesses PostgreSQL and storage services. File uploads are stored in private object storage when configured; a database byte fallback is preserved for resilience. The API can call external OCR, mapping, and optional AI services, and returns fallback output where appropriate.

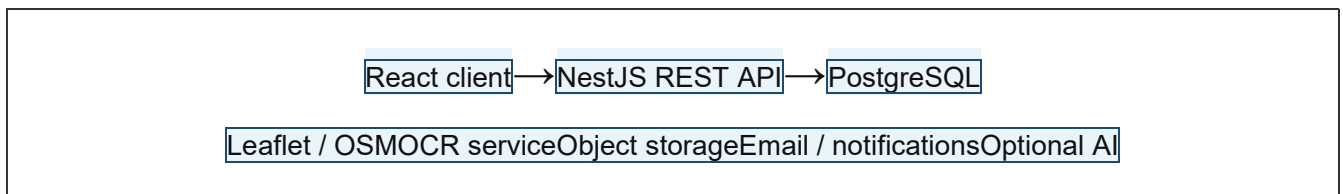


Figure 1 - High-level system architecture

3.5 Summary

The adopted stack provides a maintainable TypeScript implementation across the frontend and backend, while the integrations support the specific evidence and communication needs of the valuation process.

Chapter 4 – Analysis and Design

4.1 Introduction

This chapter presents the requirements and design of the Valuation Document Management & CRM System. The design is aligned with the implemented repositories and validates the system's intended workflow.

4.2 Functional Requirements

ID	Requirement
FR01	Visitors shall submit contact messages and valuation requests.
FR02	Users shall log in, change passwords, recover passwords, and maintain profile information.
FR03	Coordinators shall register applicants and banks, create projects and valuations, and assign technical officers.
FR04	Technical officers shall save inspection data, upload inspection forms and photos, and preserve supporting evidence.
FR05	The system shall allow officers to select GPS coordinates and record access/locality descriptions.
FR06	The system shall capture nearby comparable evidence and produce an analysis basis for market value.
FR07	The system shall support draft creation, correction, three-level review, approval, rejection reasons, and final locking.
FR08	The system shall calculate a valuation fee, support demonstration card payment or payment slips, and allow coordinator verification.
FR09	The system shall permit a bank to view only its associated report after it is locked and paid.
FR10	The system shall provide messages, notifications, and email updates for significant workflow events.

Table 3 - Functional requirements

4.3 Non-Functional Requirements

Category	Design response
Security	bcrypt password hashes; private file-storage design; sensitive configuration is kept in environment variables.
Reliability	Feature modules isolate responsibilities; uploads have a database fallback when object storage is unavailable.
Usability	Role-specific routes, shared components, form validation, cards, tables, and status views reduce unnecessary complexity.
Maintainability	Frontend and backend are organised by Home Pages, Role Pages, and Common Pages; backend services use NestJS modules.

Performance	Vite serves the development client, REST endpoints have scoped responsibilities, and PostgreSQL supports structured queries.
-------------	--

4.4 End-to-End Workflow Design

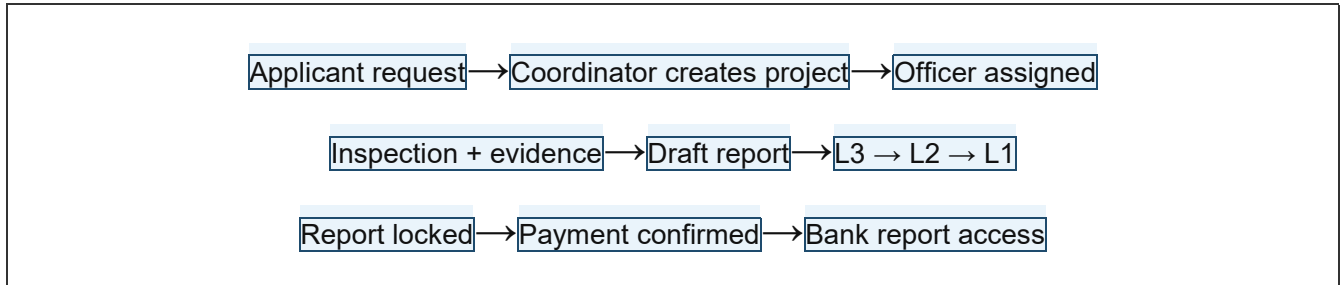


Figure 2 - End-to-end valuation workflow

4.5 Data Design

PostgreSQL provides core tables for users, projects, valuations, contact messages, valuation requests, supporting files, inspections, site photographs, map analyses, land analyses, drafts, notifications, messages, and officer leave records. The 'projects' and 'valuations' entities connect client, property, bank, officer, and valuation data. JSONB columns retain flexible form/report data while important project fields are also stored in dedicated columns for queryability.

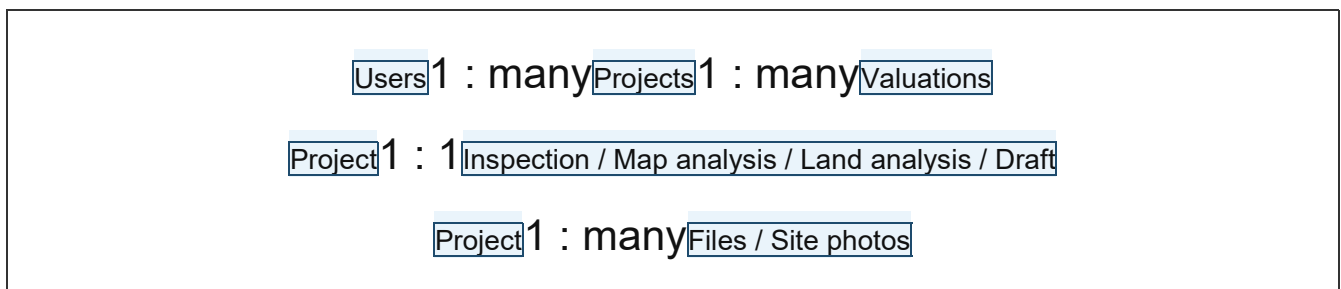


Figure 3 - Simplified entity relationships

4.6 Approval-State Design

The manager-review service defines explicit states: 'draft', 'pending_l3', 'pending_l2', 'pending_l1', and 'locked'. Rejections produce 'rejected_to_to', 'rejected_l3', or 'rejected_l2', returning the report to the preceding responsible stage. When the report is locked, the project moves to the payment stage. The report-access service permits bank viewing only if the review state is locked and payment has been confirmed.

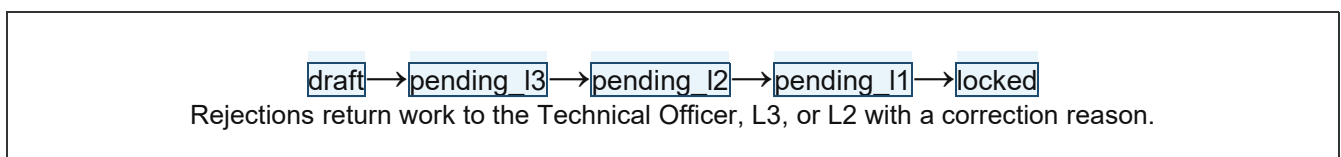


Figure 4 - Report approval state model

4.7 Summary

The design combines a relational system of record, role-specific processes, and controlled state transitions. This supports traceability from the original request through final release.

Chapter 5 – Implementation and Testing

5.1 Introduction

This chapter describes the implemented modules in the current frontend and backend codebases. It distinguishes completed functionality from capabilities that would require further production hardening.

5.2 Frontend Implementation

The frontend is implemented with React 18 and TypeScript. React Router separates public pages from the internal application shell. Public pages include the home page, services, contact, valuation request, and internal/external login. The internal layout provides dashboard, notification, profile, messaging, administration, coordinator, technical-officer, manager, applicant, and bank routes. Reusable components implement form fields, validation, cards, tables, modals, navigation, and state feedback.

5.3 Backend Implementation

The backend is a NestJS application whose root module composes feature modules for database access, mail, authentication, contact and valuation requests, coordinator functions, officer functions, report review, client/bank access, storage, notifications, messages, AI assistance, and chatbot functionality. Controllers expose REST endpoints; services implement business rules; DTOs validate input values.

5.4 Coordinator and Applicant Modules

The coordinator workflow includes applicant registration, bank registration, project creation, valuation creation, project status monitoring, technical-officer assignment, fleet management, attendance review, and payment-slip handling. The applicant can submit a public request, receive an account, upload documents, view locked project/payment information, make a demonstration card payment, or upload a bank slip for verification.

5.5 Technical-Officer Module

The technical-officer module contains assigned projects, inspection-data capture, site photo uploads, GPS mapping, description generation, nearby/comparable analysis, draft creation, and corrections. Inspection files are uploaded and passed to OCR.space; the returned text is parsed against expected labels and shown as editable values. Site photographs are stored per photo type, and an optional AI caption can be generated. Mapping saves latitude, longitude, access descriptions, locality descriptions, and the evidence used to formulate those descriptions. Nearby analysis stores comparable evidence and calculates value-related results for use in a professional report draft.

5.6 Manager, Payment and Bank Modules

Manager L3, L2, and L1 pages load reports appropriate to each stage. L3 submits to L2; L2 submits to L1; L1 locks the final report. Rejections are routed to the responsible previous stage with a reason, and notifications inform stakeholders. After lock, the fee is calculated from the

market value using a regressive band scale with a minimum fee. Payment can be marked through the demonstration card flow or held as pending when a bank slip is uploaded. A coordinator can approve or reject the slip. The bank-report endpoint confirms that the report is both locked and paid before providing access.

5.7 Testing

The following table should be completed using screenshots, API responses, and test dates from the team's final test session. The expected results are derived from the implemented workflow.

ID	Test scenario	Expected result	Final result
TC01	Submit valuation request	Request is saved and available to coordinator.	[Record result]
TC02	Invalid login	Login is rejected and no password hash is exposed.	[Record result]
TC03	Create project and valuation	Unique project/valuation data is created.	[Record result]
TC04	Assign technical officer	Assignment appears in officer's list.	[Record result]
TC05	Upload inspection form	OCR draft fields or a clear OCR error are returned.	[Record result]
TC06	Save map and analysis	Coordinates/descriptions/comparables persist for project.	[Record result]
TC07	Approval sequence	Report changes through L3, L2, L1 then locked states.	[Record result]
TC08	Reject at a manager level	Report returns to correct prior role with reason.	[Record result]
TC09	Upload payment slip	Slip is pending until coordinator verification.	[Record result]
TC10	Bank access before payment	Bank cannot view report.	[Record result]
TC11	Bank access after payment	Associated bank can view locked, paid report.	[Record result]

Table 4 - Final testing checklist

5.8 Summary

The implemented modules cover the primary business flow from request to controlled report access. The next chapter evaluates the delivered solution and its remaining limitations.

Chapter 6 – Discussion and Conclusion

6.1 Discussion

The implemented system converts a fragmented set of valuation activities into a connected process. It preserves the major hand-offs between applicant, coordinator, technical officer, managers, and bank. The technical-officer area is particularly significant because it brings inspection, file evidence, photographs, mapping, comparable analysis, and report drafting together under a single project context. The approval state model provides a clear mechanism for quality review and correction before finalisation.

The use of OCR and optional AI assistance should be understood as productivity support rather than an autonomous valuation decision. OCR output is editable, AI narrative has fallbacks, and managers retain approval responsibility. This design reduces routine transcription work while preserving professional accountability.

6.2 Achievements

- A working React/NestJS/PostgreSQL application is organised into role-specific frontend pages and backend feature modules.
- The solution supports public requests, authentication, coordinator project management, technical evidence capture, multi-level approval, payment flow, notifications, and controlled bank access.
- It integrates private storage with fallback behaviour, providing resilience for important uploaded evidence.
- It incorporates map and comparable-evidence functionality that supports a documented valuation basis.
- It provides a structured, traceable report lifecycle through defined review and payment states.

6.3 Limitations

- The card-payment operation is explicitly a demonstration flow and is not a live payment-gateway integration.
- External OCR, mapping, and optional AI features depend on third-party availability, rate limits, and configuration.
- OCR parsing is currently configured for English output and requires careful human verification, especially for handwriting and multilingual documents.
- Production deployment should add comprehensive API-level authorisation guards, audit logging, rate limiting, monitoring, backup procedures, and security review.
- The final report should include measured usability and performance-test evidence; claims of time saving should not be made without recorded study data.

6.4 Future Improvements

1. Integrate a PCI-compliant payment gateway and transaction reconciliation.
2. Implement fine-grained backend role permissions, immutable audit trails, and report digital signatures.

3. Add production monitoring, automated backups, CI/CD, vulnerability scanning, and disaster recovery procedures.
4. Add field-friendly mobile/offline inspection capture and richer evidence provenance.
5. Extend reporting dashboards for workload, turnaround time, and valuation trends.
6. Conduct formal user acceptance testing with technical officers, managers, applicants, and bank officers.

6.5 Conclusion

The Valuation Document Management & CRM System provides a practical digital workflow for a valuation company. It supports the journey from the first request through project coordination, evidence collection, report preparation, managerial review, payment confirmation, and secure bank visibility. By integrating these processes in a modular web platform, the project improves operational transparency and establishes a strong foundation for future production-grade enhancements.

References

- [1] PostgreSQL Global Development Group, *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/>
- [2] React Team, *React Documentation*. [Online]. Available: <https://react.dev/>
- [3] NestJS, *NestJS Documentation*. [Online]. Available: <https://docs.nestjs.com/>
- [4] Leaflet, *Leaflet Documentation*. [Online]. Available: <https://leafletjs.com/>
- [5] OpenStreetMap Foundation, *Nominatim Documentation*. [Online]. Available: <https://nominatim.org/>
- [6] OWASP Foundation, *OWASP Authentication Cheat Sheet*. [Online]. Available: <https://cheatsheetseries.owasp.org/>
- [7] OCR.space, *OCR API Documentation*. [Online]. Available: <https://ocr.space/ocrapi>
- [8] International Valuation Standards Council, *International Valuation Standards*. [Online]. Available: <https://www.ivsc.org/standards/>

Verify these references and add any client-provided, academic, and professional sources that the team actually consulted. Follow the final citation style required by the module.

Appendix A – Individual Contribution to the Project

Member	Contribution area
G.C.G. Jagoda (235051C)	Project coordination, project-board management, SRS and report preparation, use-case/activity design, research, UI design support, and repository setup.
S.W.P. Methmini (235075D)	Sequence and ER design, requirements clarification, report information gathering, shared/common implementation support, and Bank/Loan Applicant area contribution.
A.L.A.A. Shanali (235104T)	Manager L1/L2/L3 area contribution, presentation preparation, UI design support, meeting documentation, research, and timeline work.
H.M.R.R. Narampanawa (235078M)	Coordinator area contribution, use-case/activity/ER/sequence design, UI design support, source gathering, and technology-requirement analysis.
B.G.R. Dilsara (235030L)	Technical-officer area contribution, activity/class/ER/use-case design, UI design support, research, and project-timeline work.

Each member should review and amend this table to accurately reflect final contributions before submission.

Appendix B – Evidence and Screenshot Checklist

Insert clear, numbered screenshots from the completed running system before submission. Mask all passwords, API keys, database URLs, personal data, and payment

references.

1. Home page and public valuation-request form.
2. Internal and external login screens.
3. Coordinator applicant/bank registration and project creation.
4. Technical-officer assignment and assigned project list.
5. Inspection form upload, OCR response, and saved inspection data.
6. Site-photo upload screen.
7. GPS map with selected location and generated descriptions.
8. Nearby comparable-analysis screen.
9. Draft report editor.
10. Manager L3, L2, and L1 approval/correction screens.
11. Applicant payment and payment-slip submission screen.
12. Coordinator payment-slip verification screen.
13. Bank report-access screen after payment.

System Setup

Backend: ``cd Land-Valuation-Backend`, `npm install`, then `npm run start:dev`.`

Frontend: ``cd Land-Valuation-Frontend`, `npm install`, then `npm run dev`.`

The frontend development server is configured for port 3000 and proxies ``/api`` requests to the NestJS backend on port 4000.